

第8章 网络通信

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

本章学习目标

- 建立 C 的 socket API 到 Go net 包的完整对照,看懂 `htons` 时代的终结
- 掌握 TCP 编程:服务端(`goroutine per connection`)与客户端完整示例
- 掌握 UDP 的无连接语义与适用场景
- 掌握超时与优雅关闭:`SetDeadline`、`Close`、`context` 取消
- 掌握 HTTP 客户端:`GET/POST/JSON` 的完整姿势(含超时与连接复用)
- 掌握 HTTP 服务端:标准库最小服务器、路由注册、JSON 响应
- 理解粘包/拆包问题,掌握"长度前缀"协议设计与实现
- 了解生产坑:连接泄漏、`TIME_WAIT`、优雅停机(`signal + Shutdown`)

从 C 到 Go:本章主题如何迁移

C 的联网是教科书式的原始: `socket()`、`bind()`、`listen()`、`accept()` 一套流程, `htons()` / `htonl()` 手工处理字节序, `struct sockaddr` 的字段还要自己填;阻塞模型下每个连接一个线程,线程切换开销让"万级连接"成为架构挑战;HTTP 更没有标准库,要么手写要么引第三方。

Go 的 `net` 包把这些全部折叠成两个函数: `net.Dial` (客户端连接)与 `net.Listen + Accept` (服务端监听)。地址直接写 `"127.0.0.1:8080"` 字符串,字节序由标准库处理;并发模型是 **goroutine per connection**——每来一个连接开一个几 KB 的 `goroutine`,十万并发连接是日常,这正是第 5、9 章内存与并发知识的总应用。

HTTP 更是标准库的强项: `net/http` 一个包同时提供客户端(带连接池、重试、超时)与服务端(路由、中间件、优雅停机)。C 里要写几千行的东西,Go 里几十行,而且生产可用。协议层(粘包/拆包)仍然是你的职责——TCP 是字节流,没有消息边界,本章会给出三种协议设计并完整实现长度前缀版。

一句话主线: C 的 `socket` 是"系统调用 + 手工组装",Go 的 `net` 是"高并发 + 零样板"——语义相同(仍是 TCP/UDP/端口),体力活全部消失。

前置知识自查

- TCP 三次握手/四次挥手、UDP 无连接、端口、IP 地址
- C 的 `socket/bind/listen/accept/connect/send/recv` 流程与 `htons/htonl`
- 阻塞与非阻塞 I/O、`select/epoll` 的概念(了解即可)
- 第 5 章的 `goroutine` 泄漏、第 9 章的 `channel`(本章会用到基础)
- 第 7 章的 `encoding/binary` 与 `bufio`(协议实现直接使用)

🚀 本章学习路线

1. **第一阶段**:§8.1–§8.3,TCP 服务端 + 客户端亲手跑通(这是最重要的 30 行代码)
2. **第二阶段**:§8.4–§8.6,超时与 HTTP 两个主题,每个都"抄进生产直接能用"
3. **第三阶段(实验)**:§8.7–§8.8 与章末实验:带长度前缀协议的回声服务器 + 客户端
4. **验收标准**:能不看资料写出"服务端 Accept 循环 + goroutine 处理 + 超时 + 优雅退出"骨架

本章知识导图

- §8.1 API 对照:C 的 socket 全家桶 → Go 的 net.Dial/Listen
- §8.2 TCP 编程:服务端(goroutine per connection)与客户端
- §8.3 UDP 编程:无连接语义与适用场景
- §8.4 超时与优雅关闭:SetDeadline 与 Close
- §8.5 HTTP 客户端:GET/POST/JSON、超时与连接复用
- §8.6 HTTP 服务端:标准库路由、JSON 响应、优雅停机
- §8.7 粘包/拆包:定长、分隔符、长度前缀三种协议
- §8.8 生产坑:连接泄漏、TIME_WAIT、资源与日志

§8.1 API 对照:从 socket 到 net 包

C 的写法	Go 的写法
socket(AF_INET, SOCK_STREAM, 0)	不需要:net.Dial 里隐式创建
htons(port); bind(fd, &addr, len)	net.Listen("tcp", "127.0.0.1:8080"),地址写字符串
listen(fd, backlog); accept(fd, ...)	ln.Accept() → (net.Conn, error)
connect(fd, ...)	net.Dial("tcp", "host:port") → (net.Conn, error)
send/recv(fd, buf, len, 0)	conn.Write(buf) / conn.Read(buf),返回 (n, err)
close(fd)	conn.Close() (配合 defer)
每连接一线程(或 select 状态机)	每连接一个 goroutine,几 KB 开销,十万并发是日常
getsockopt/setsockopt 调超时	conn.SetDeadline(t) / SetReadDeadline / SetWriteDeadline

★ 重点:net.Conn 是 io.Reader + io.Writer

net.Conn 同时实现 io.Reader 与 io.Writer ——第 6 章"接口定义在使用方"的完美例证:io.Copy(dst, src)、bufio.NewScanner(conn)、json.NewDecoder(conn) 全部直接可用。网络连接在 Go 里就是"另一个文件",你不需要单独的 socket API。

§8.2 TCP 编程:goroutine per connection

示例 8-1 TCP 服务端:每连接一个 goroutine

```
package main

import (
    "fmt"
    "io"
    "net"
)

// handleConn 每个连接一个 goroutine:把收到的字节原样发回
func handleConn(conn net.Conn) {
    defer conn.Close() // 处理完必关连接
    fmt.Println("新连接:", conn.RemoteAddr())
    n, err := io.Copy(conn, conn) // 读多少写多少:回声
    fmt.Printf("连接 %s 结束,传输 %d 字节,err=%v\n",
        conn.RemoteAddr(), n, err)
}

func main() {
    // Listen 一步完成 socket+bind+listen
    ln, err := net.Listen("tcp", "127.0.0.1:8080")
    if err != nil {
        fmt.Println("监听失败:", err)
        return
    }
    defer ln.Close()
    fmt.Println("服务端监听 127.0.0.1:8080")

    for {
        conn, err := ln.Accept()
        if err != nil {
            fmt.Println("Accept 错误:", err)
            continue // 临时错误继续接受
        }
        go handleConn(conn) // goroutine per connection
    }
}
```

示例 8-2 TCP 客户端

```

package main

import (
    "fmt"
    "net"
    "time"
)

func main() {
    // Dial 一步完成 socket+connect
    conn, err := net.Dial("tcp", "127.0.0.1:8080")
    if err != nil {
        fmt.Println("连接失败:", err)
        return
    }
    defer conn.Close()

    // 必须设置超时,防止永久挂起
    conn.SetDeadline(time.Now().Add(5 * time.Second))

    msg := "你好,服务器"
    n, err := conn.Write([]byte(msg))
    if err != nil {
        fmt.Println("发送失败:", err)
        return
    }
    fmt.Printf("已发送 %d 字节\n", n)

    buf := make([]byte, 1024)
    n, err = conn.Read(buf)
    if err != nil {
        fmt.Println("接收失败:", err)
        return
    }
    fmt.Printf("收到回声: %s\n", string(buf[:n]))
}

```

⚠ 易错点 8-1:Accept 循环里的错误处理

Accept 失败不一定要退出:临时错误(如 EMFILE 句柄耗尽)应 `continue` 重试,只有永久性错误(监听器已关闭)才退出。判断方式: `if errors.Is(err, net.ErrClosed)` 说明监听器主动关闭,此时 `return`;否则 `sleep` 几十毫秒继续。生产服务这段代码要写得比示例更稳。

§8.3 UDP 编程:无连接

UDP 无连接:客户端 `DialUDP` 或直接 `net.Dial("udp", ...)` 后 `Read/Write`;服务端 `ListenUDP` + `ReadFromUDP` 接收, `WriteToUDP` 回复。不保证到达、不保证顺序、不保证不重复——适合丢得起包、要低延迟的场景(游戏状态同步、DNS、日志采集)。

示例 8-3 UDP 服务端与客户端

```

package main

import (
    "fmt"
    "net"
    "time"
)

// udpServer 监听 9090,收到消息原样回复给发送方
func udpServer() {
    addr, _ := net.ResolveUDPAddr("udp", "127.0.0.1:9090")
    conn, err := net.ListenUDP("udp", addr)
    if err != nil {
        fmt.Println("UDP 监听失败:", err)
        return
    }
    defer conn.Close()
    fmt.Println("UDP 服务端监听 127.0.0.1:9090")

    buf := make([]byte, 2048)
    for {
        n, remote, err := conn.ReadFromUDP(buf)
        if err != nil {
            fmt.Println("读取失败:", err)
            return
        }
        // WriteToUDP:发给"刚收到消息的那个地址"
        _, err = conn.WriteToUDP(buf[:n], remote)
        if err != nil {
            fmt.Println("回复失败:", err)
        }
    }
}

func main() {
    go udpServer()
    time.Sleep(200 * time.Millisecond)

    // 客户端:Dial 后像 TCP 一样读写
    conn, err := net.Dial("udp", "127.0.0.1:9090")
    if err != nil {
        fmt.Println("UDP 连接失败:", err)
        return
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(3 * time.Second))

    _, _ = conn.Write([]byte("UDP 你好"))
    buf := make([]byte, 2048)
    n, err := conn.Read(buf)
    if err != nil {
        fmt.Println("UDP 接收失败:", err)
        return
    }
}

```

```
    fmt.Printf("UDP 回声: %s\n", string(buf[:n]))
}
```

§8.4 超时与优雅关闭

为什么必须设置超时:生产事故最常见形态——对端半开连接、网络分区、对端进程崩溃, `Read` 可能永久阻塞,goroutine 与连接一起泄漏。 `SetDeadline` 之后,超时返回 `net.Error, Timeout()` 为 `true`,与业务错误区分。

示例 8-4 超时处理与连接复用

```
package main

import (
    "fmt"
    "net"
    "time"
)

// readWithTimeout 带超时读取:超时后返回可区分的错误
func readWithTimeout(conn net.Conn) ([]byte, error) {
    conn.SetReadDeadline(time.Now().Add(2 * time.Second))
    buf := make([]byte, 4096)
    n, err := conn.Read(buf)
    if err != nil {
        if netErr, ok := err.(net.Error); ok && netErr.Timeout() {
            return nil, fmt.Errorf("读取超时(2s): %w", err)
        }
        return nil, err
    }
    return buf[:n], nil
}

func main() {
    ln, _ := net.Listen("tcp", "127.0.0.1:8081")
    defer ln.Close()

    // 服务端:不回复,故意让客户端超时
    go func() {
        conn, _ := ln.Accept()
        time.Sleep(10 * time.Second) // 挂起,不读不写
        conn.Close()
    }()

    conn, _ := net.Dial("tcp", "127.0.0.1:8081")
    defer conn.Close()
    _, _ = conn.Write([]byte("ping"))

    if _, err := readWithTimeout(conn); err != nil {
        fmt.Println(err) // 2 秒后返回"读取超时"
    }
}
```

工程建议:超时的三层设置

① **连接建立超时** : `net.DialTimeout("tcp", addr, 3*time.Second)`; ② **单次读写超时**: `SetReadDeadline/SetWriteDeadline`(读、写分开设,不要只用一个 `SetDeadline`); ③ **空闲超时**: 服务端用 `SetKeepAlive` + 心跳/读超时踢掉僵尸连接。HTTP 客户端还有第 4 层: `http.Client.Timeout`(§8.5)。

§8.5 HTTP 客户端:GET/POST/JSON

`net/http` 的客户端默认行为比 C 的任何手写实现都完整:连接池、Keep-Alive、重定向、gzip、超时。但有一个默认值是陷阱:**`http.DefaultClient` 没有超时**——生产必须显式配置。

示例 8-5 HTTP 客户端:GET 与 JSON

```

package main

import (
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "time"
)

// 生产客户端:必须配置超时与连接池
var client = &http.Client{
    Timeout: 5 * time.Second, // 总超时(含连接、重定向、读体)
    Transport: &http.Transport{
        MaxIdleConns:      100,
        MaxIdleConnsPerHost: 20, // 每个主机保持的空闲连接数
        IdleConnTimeout:    90 * time.Second,
    },
}

type repo struct {
    FullName string `json:"full_name"`
    Stars    int    `json:"stargazers_count"`
}

func main() {
    // GET 请求
    resp, err := client.Get("https://api.github.com/repos/golang/go")
    if err != nil {
        fmt.Println("GET 失败:", err)
        return
    }
    defer resp.Body.Close() // 响应体必须关闭,否则连接不归还池

    fmt.Println("状态码:", resp.StatusCode)

    // 直接 JSON 解码
    var r repo
    if err := json.NewDecoder(resp.Body).Decode(&r); err != nil {
        fmt.Println("解析失败:", err)
        return
    }
    fmt.Printf("%s 有 %d 颗星\n", r.FullName, r.Stars)

    // 幂等读走 io.ReadAll 也行(小响应)
    body, _ := io.ReadAll(resp.Body)
    fmt.Println("响应体长度:", len(body))
}

```

| 示例 8-6 HTTP 客户端:POST JSON


```

package main

import (
    "bytes"
    "encoding/json"
    "fmt"
    "net/http"
)

type reqBody struct {
    Name string `json:"name"`
    Score int    `json:"score"`
}

func main() {
    body, _ := json.Marshal(reqBody{Name: "张三", Score: 92})

    // 注意:1.20+ 可以用 http.NewRequestWithContext
    resp, err := http.Post(
        "https://httpbin.org/post",
        "application/json",
        bytes.NewReader(body),
    )
    if err != nil {
        fmt.Println("POST 失败:", err)
        return
    }
    defer resp.Body.Close()

    // 把响应原样打印(演示用)
    var out map[string]any
    _ = json.NewDecoder(resp.Body).Decode(&out)
    fmt.Println("服务端返回 data:", out["data"])
}

```

⚠ 易错点 8-2:不关 `resp.Body` 导致连接泄漏

`http.Client` 复用连接的机制要求响应体被读完并关闭;不 `Close` 或不读完,连接无法归还连接池,最终耗尽文件描述符。铁律: `resp, err := ...` 之后**立刻** `defer resp.Body.Close()` ,再处理业务。

§8.6 HTTP 服务端:标准库最小服务器

示例 8-7 最小 HTTP 服务端(约 30 行)

```

package main

import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "time"
)

type helloResp struct {
    Message string `json:"message"`
    Time    time.Time `json:"time"`
}

// 处理器:http.HandlerFunc 即"函数即处理器"
func hello(w http.ResponseWriter, r *http.Request) {
    // 方法检查(路由本身不管 HTTP 方法)
    if r.Method != http.MethodGet {
        http.Error(w, "只支持 GET", http.StatusMethodNotAllowed)
        return
    }
    resp := helloResp{Message: "你好,Go!", Time: time.Now()}
    w.Header().Set("Content-Type", "application/json; charset=utf-8")
    _ = json.NewEncoder(w).Encode(resp)
}

func main() {
    // 路由注册:pattern 带路径与方法(GO 1.22+ 支持 "GET /hello")
    http.HandleFunc("/hello", hello)
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "页面: %s\n", r.URL.Path)
    })

    // 生产要点:Server 结构体显式配置超时,不要裸 ListenAndServe
    srv := &http.Server{
        Addr:           "127.0.0.1:8082",
        ReadTimeout:    5 * time.Second, // 读请求超时
        WriteTimeout:   10 * time.Second, // 写响应超时
        IdleTimeout:    60 * time.Second, // 空闲连接超时
    }
    log.Println("HTTP 服务启动 127.0.0.1:8082")
    log.Fatal(srv.ListenAndServe())
}

```

★ 重点:生产框架方向(Gin 一句话)

标准库能覆盖大量生产需求(路由、JSON、优雅停机),但复杂 API 项目通常会选框架:**Gin** 是当前事实标准——路由参数、中间件、绑定校验、性能俱佳; `go get github.com/gin-gonic/gin` 即可开始。本书所有 HTTP 知识在 Gin 里 1:1 复用(handler 概念、JSON 编解码、上下文)。

§8.7 粘包/拆包:协议设计

问题:TCP 是字节流,不保消息边界。 `send("AB"); send("CD")` 对端可能一次读到 "ABCD"(粘包),也可能分两次读到 "A" "BCD"(拆包)。三种协议设计:

- **定长协议:**每条消息固定 N 字节,不足补零。简单,但浪费带宽、不支持变长内容
- **分隔符协议:**每条消息以 `\n` (或特殊字符)结尾,如 HTTP 头、Redis RESP。注意内容里不能出现分隔符(需要转义)
- **长度前缀协议:**每条消息 = 2/4 字节长度 + 内容,如 HTTP Content-Length、Minecraft 协议。**最通用,推荐**

示例 8-8 长度前缀协议:编解码

```

package main

import (
    "encoding/binary"
    "fmt"
    "io"
    "strings"
)

// writeMsg 发送:2 字节长度(小端)+ 消息体
func writeMsg(w io.Writer, msg []byte) error {
    if len(msg) > 65535 {
        return fmt.Errorf("消息过长: %d", len(msg))
    }
    header := make([]byte, 2)
    binary.LittleEndian.PutUint16(header, uint16(len(msg)))
    if _, err := w.Write(header); err != nil {
        return err
    }
    _, err := w.Write(msg)
    return err
}

// readMsg 读取:先读满 2 字节长度,再读满消息体
func readMsg(r io.Reader) ([]byte, error) {
    header := make([]byte, 2)
    if _, err := io.ReadFull(r, header); err != nil {
        return nil, err // 注意:不能用 Read,Read 可能只读到 1 字节
    }
    n := binary.LittleEndian.Uint16(header)
    body := make([]byte, n)
    _, err := io.ReadFull(r, body)
    return body, err
}

func main() {
    // 模拟"两条消息一次到达"(粘包):把两条消息写进同一个缓冲区
    var buf strings.Builder
    // 实际网络场景用 net.Conn 代替;这里用内存模拟
    _ = writeMsg(&buf, []byte("第一条"))
    _ = writeMsg(&buf, []byte("第二条消息"))

    // 按协议边界拆回两条
    reader := strings.NewReader(buf.String())
    for {
        msg, err := readMsg(reader)
        if err == io.EOF {
            break
        }
        fmt.Printf("拆出消息: %q\n", msg)
    }
}

```

⚠ 易错点 8-3:Read 一次不一定读满,必须 io.ReadFull

网络 Read 的返回 n 可以小于请求长度(拆包!),处理"先读 N 字节"时必须循环或用 `io.ReadFull` (内部循环直到读满或 EOF/错误)。手写 `conn.Read(buf)` 一次就当作"读到了整条消息",是协议解析最常见的 bug。

§8.8 生产坑:连接泄漏、TIME_WAIT 与优雅停机

- **连接泄漏**:每 accept 一个连接就开 goroutine 且不 Close,连接与 goroutine 成对泄漏。关闭义务在"处理方",defer Close 必须紧跟 Accept/Dial
- **TIME_WAIT**:主动关闭的一方会进入 TIME_WAIT(约 2 分钟),大量短连接客户端会在高并发下耗尽本地端口。对策:服务端不要主动关(让客户端关)、连接复用(HTTP 的 Keep-Alive/连接池)、调大 ephemeral 端口范围
- **优雅停机**:收到 SIGINT/SIGTERM 时,先停止接受新连接,再等存量请求完成(`http.Server.Shutdown(ctx)`),最后退出
- **字节序与协议版本**:跨语言互通时,长度字段一律固定宽度 + 显式字节序;协议留版本号字段

示例 8-9 优雅停机:sigint 触发 Shutdown

```

package main

import (
    "context"
    "fmt"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"
)

func main() {
    srv := &http.Server{Addr: "127.0.0.1:8083"}

    go func() {
        fmt.Println("服务启动")
        if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
            fmt.Println("服务异常:", err)
        }
    }()

    // 等待退出信号
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, os.Interrupt, syscall.SIGTERM)
    <-quit
    fmt.Println("收到退出信号,优雅停机...")

    // 给存量请求 10 秒完成,超时强制关闭
    ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()
    if err := srv.Shutdown(ctx); err != nil {
        fmt.Println("强制关闭:", err)
    }
    fmt.Println("已退出")
}

```

本章速查表

主题	要点
监听	<code>net.Listen("tcp", "addr:port")</code> → <code>net.Listener</code> ; <code>Accept()</code> 循环
连接	<code>net.Dial("tcp", ...)</code> / <code>net.DialTimeout</code> ; UDP 同 API
读写	<code>conn.Read/Write</code> 返回 <code>(n, err)</code> ; <code>net.Conn</code> 即 <code>io.ReadWriter</code>
并发模型	goroutine per connection: <code>Accept</code> 后 <code>go handleConn(conn)</code>
关闭	<code>defer conn.Close()</code> 紧跟建立; 不关 = 泄漏
UDP	<code>ListenUDP</code> / <code>ReadFromUDP</code> / <code>WriteToUDP</code> ; 无连接, 丢包不保证
超时	<code>SetReadDeadline/SetWriteDeadline</code> ; <code>net.Error.Timeout()</code> 判断
HTTP 客户端	自定义 <code>http.Client(Timeout+Transport)</code> ; 响应体必 <code>defer Close</code>
HTTP 服务端	<code>http.HandleFunc</code> + <code>http.Server(Read/Write/IdleTimeout)</code>
JSON 传输	<code>json.NewEncoder(w).Encode</code> / <code>json.NewDecoder(r.Body).Decode</code>
粘包拆包	定长 / 分隔符 / 长度前缀; 推荐长度前缀 + <code>io.ReadFull</code>
优雅停机	<code>signal.Notify</code> + <code>srv.Shutdown(ctx)</code> ; <code>ErrServerClosed</code> 视为正常
生产框架	Gin(路由/中间件/绑定); 标准库仍是基础

最小必会清单

- 能默写 TCP 服务端骨架: `Listen` → `Accept` 循环 → `go handleConn` → `defer Close`
- 能默写 TCP 客户端骨架: `Dial` → `SetDeadline` → `Write/Read` → `Close`
- 能解释 UDP 与 TCP 的适用场景差异, 写出 UDP 回显两端
- 能解释"为什么必须设超时", 写出 `SetReadDeadline` 与 `Timeout()` 判断
- 能写出带超时与连接池配置的 `http.Client`, GET + JSON 解码完整流程
- 能写出 `http.Server` 的显式超时配置与 `HandleFunc` 注册
- 能解释粘包/拆包的成因, 写出长度前缀协议的 `writeMsg/readMsg`
- 能解释"为什么必须用 `io.ReadFull` 而不是 `Read`"

自测题

Q 1. C 的 `accept()` 返回什么? Go 的 `ln.Accept()` 返回什么? 两者的错误处理有何不同?

✅ 参考答案

C 返回文件描述符(int), 失败返回 -1 并设 `errno`; Go 返回 `(net.Conn, error)`, 失败时 `err` 携带原因(如监听器关闭时 `errors.Is(err, net.ErrClosed)`)。Go 的 `Accept` 循环对临时错误应 `continue` 重试, 对关闭错误应退出。

Q 2. 服务端用 `io.Copy(conn, conn)` 实现回声, 为什么可行? 它替代了 C 的什么循环?

✅ 参考答案

net.Conn 同时实现 io.Reader 与 io.Writer,io.Copy 内部循环 Read→Write 直到 EOF,替代了 C 的 `while ((n = recv(...)) > 0) send(...)` 手写循环。对端关闭(读到 EOF)时 io.Copy 返回 nil 错误,此时应关闭连接。

Q 3. 客户端两次 `Write("AB")`、`Write("CD")`,服务端可能读到什么?协议如何解决?

✅ 参考答案

可能一次读到 "ABCD"(粘包),或 "A"+"BCD"、"AB"+"CD"、"ABC"+"D"(拆包)等任意切分。解决:定长、分隔符或长度前缀协议。长度前缀:先读 2 字节长度 n,再 io.ReadFull 读 n 字节内容,保证每次拆出完整消息。

Q 4. 为什么 http.DefaultClient 不适合生产?写出正确的客户端配置要点。

✅ 参考答案

DefaultClient 没有 Timeout,任何一次请求都可能无限挂起(连接建立、读响应),导致 goroutine 与连接泄漏。要点:设置 Timeout;配置 Transport 的 MaxIdleConnsPerHost 与 IdleConnTimeout;响应体 defer Close;长任务用 http.NewRequestWithContext 配合 context 超时(第 9 章)。

Q 5. 简述优雅停机的完整步骤与所用 API。

✅ 参考答案

①signal.Notify 捕获 SIGINT/SIGTERM;②收到信号后调用 `srv.Shutdown(ctx)` (先停止接受新连接,再等待存量请求完成);③ ctx 带超时,超时后 Shutdown 返回错误,可强制 Close;④ ListenAndServe 返回 `http.ErrServerClosed` 视为正常退出,不是错误。

Q 6. UDP 服务端如何知道"把回复发给谁"?

✅ 参考答案

ReadFromUDP 的第二个返回值是发送方地址:`conn.ReadFromUDP(buf)` 返回 (n, remoteAddr),回复用 `conn.WriteToUDP(buf[:n], remoteAddr)`。因为 UDP 无连接,每个数据报自带源地址,服务端必须把地址随消息一起传递。

章末实验题:带协议的回声服务器与客户端

实验 8:回声服务器 + 长度前缀协议 + 优雅停机

实验目标:实现一个生产形态的回声服务器:长度前缀协议解决粘包、goroutine per connection、读写超时、优雅停机;客户端发送多条消息并校验回声。

实验要求:

- 任务 1:实现 `writeMsg / readMsg`,用 `io.ReadFull` 保证读满(覆盖:\$8.7 长度前缀协议)
- 任务 2:服务端 Listen + Accept 循环,每连接一个 goroutine(覆盖:\$8.2 并发模型)
- 任务 3:每个连接设置读写超时,空闲连接被踢出(覆盖:\$8.4 超时)
- 任务 4:对端断开(EOF)时正常退出 goroutine,不泄漏(覆盖:\$8.2、第 5 章)
- 任务 5:signal.Notify + 关闭监听器,优雅退出(覆盖:\$8.8 优雅停机)
- 任务 6:客户端连续发送多条消息并逐条校验回声内容一致(覆盖:\$8.2 客户端)
- 任务 7:消息内容包含中文与长文本,验证长度字段正确处理多字节(覆盖:\$7.4 二进制)
- 任务 8:用 `go vet` 与 `go run -race` 验证无竞态(覆盖:第 9 章预告)

实验环境:Go 1.21+;分别以 `go run main.go` (服务端)与 `go run main.go client` (客户端)运行。

第一步 写协议层 `writeMsg/readMsg`,并用内存缓冲(如 `strings.Reader`)单测拆包逻辑

第二步 服务端:Accept 循环 + `go handleConn`;handleConn 里循环 `readMsg` → 打印 → `writeMsg` 回发

第三步 超时:SetReadDeadline 每次读前设置;超时错误打印后退出 goroutine

第四步 客户端:循环发送 ["hello", "世界你好", 长文本],每发一条读一条并比对

第五步 后加优雅停机:主 goroutine 等信号,收到后 `ln.Close()`,Accept 返回 `ErrClosed` 退出循环

✔ 参考答案要点

```

// 回声服务器(长度前缀协议)+ 客户端:覆盖第 8 章全部知识点
package main

import (
    "bufio"
    "encoding/binary"
    "fmt"
    "io"
    "net"
    "os"
    "os/signal"
    "strings"
    "syscall"
    "time"
)

const addr = "127.0.0.1:9876"

// writeMsg 发送:2 字节小端长度 + 消息体($8.7 长度前缀)
func writeMsg(w io.Writer, msg []byte) error {
    if len(msg) > 65535 {
        return fmt.Errorf("消息过长: %d 字节", len(msg))
    }
    header := make([]byte, 2)
    binary.LittleEndian.PutUint16(header, uint16(len(msg)))
    if _, err := w.Write(header); err != nil {
        return err
    }
    _, err := w.Write(msg)
    return err
}

// readMsg 读取:io.ReadFull 保证读满,解决拆包($8.7)
func readMsg(r io.Reader) ([]byte, error) {
    header := make([]byte, 2)
    if _, err := io.ReadFull(r, header); err != nil {
        return nil, err // 对端关闭:io.EOF
    }
    n := binary.LittleEndian.Uint16(header)
    if n > 4096 {
        return nil, fmt.Errorf("非法消息长度: %d", n)
    }
    body := make([]byte, n)
    _, err := io.ReadFull(r, body)
    return body, err
}

// handleConn 每连接一个 goroutine:循环读→回发($8.2)
func handleConn(conn net.Conn) {
    defer conn.Close() // 关闭义务在处理方($8.8)
    fmt.Println("新连接:", conn.RemoteAddr())
    reader := bufio.NewReader(conn) // 缓冲包装,减少系统调用
    for {
        // 每次读前刷新超时:空闲踢出($8.4)
        conn.SetReadDeadline(time.Now().Add(30 * time.Second))
    }
}

```

```

    msg, err := readMsg(reader)
    if err != nil {
        if netErr, ok := err.(net.Error); ok && netErr.Timeout() {
            fmt.Println("连接空闲超时,关闭:", conn.RemoteAddr())
        } else if err != io.EOF {
            fmt.Println("读取错误:", err)
        }
        return // 对端断开或超时:退出 goroutine,不泄漏($5.5)
    }
    fmt.Printf("收到 %q,原样回发\n", msg)
    if err := writeMsg(conn, msg); err != nil {
        fmt.Println("回发失败:", err)
        return
    }
}
}

// runServer 服务端:Accept 循环 + 优雅停机($8.8)
func runServer() {
    ln, err := net.Listen("tcp", addr)
    if err != nil {
        fmt.Fprintln(os.Stderr, "监听失败:", err)
        os.Exit(1)
    }
    fmt.Println("服务端监听:", addr)

    // Accept 循环放 goroutine,主 goroutine 等信号
    acceptDone := make(chan struct{})
    go func() {
        defer close(acceptDone)
        for {
            conn, err := ln.Accept()
            if err != nil {
                if strings.Contains(err.Error(), "closed") || errorsIsClosed(err) {
                    return // 监听器已关闭:正常退出
                }
                fmt.Println("Accept 错误:", err)
                time.Sleep(100 * time.Millisecond)
                continue
            }
            go handleConn(conn) // goroutine per connection
        }
    }()

    // 等待 SIGINT/SIGTERM
    sig := make(chan os.Signal, 1)
    signal.Notify(sig, os.Interrupt, syscall.SIGTERM)
    <-sig
    fmt.Println("收到退出信号,停止接受新连接")
    ln.Close()
    <-acceptDone // 等 Accept 循环退出
    fmt.Println("服务已退出")
}

// errorsIsClosed 判断监听器是否已关闭(避免引入 errors 包依赖)

```

```

func errorsIsClosed(err error) bool {
    return err == net.ErrClosed
}

// runClient 客户端:发送多条消息并校验回声($8.2)
func runClient() {
    conn, err := net.Dial("tcp", addr)
    if err != nil {
        fmt.Fprintln(os.Stderr, "连接失败:", err)
        os.Exit(1)
    }
    defer conn.Close()
    conn.SetDeadline(time.Now().Add(10 * time.Second)) // 总超时($8.4)

    msgs := []string{
        "hello",
        "世界你好",
        strings.Repeat("长文本", 50), // 超长消息:验证长度字段
    }
    reader := bufio.NewReader(conn)
    for _, m := range msgs {
        if err := writeMsg(conn, []byte(m)); err != nil {
            fmt.Println("发送失败:", err)
            return
        }
        echo, err := readMsg(reader)
        if err != nil {
            fmt.Println("接收失败:", err)
            return
        }
        ok := "一致"
        if string(echo) != m {
            ok = "不一致!"
        }
        fmt.Printf("消息(%d 字节)回声%s: %q\n", len(m), ok, echo)
    }
    fmt.Println("客户端测试完成")
}

func main() {
    // 通过命令行参数选择角色($7.7)
    if len(os.Args) > 1 && os.Args[1] == "client" {
        runClient()
        return
    }
    runServer()
}

```

设计说明:实验代码是本章知识的总装:协议层独立成函数(可单测),handleConn 体现了"关闭义务在处理方 + 超时踢出 + EOF 正常退出"的完整生命周期,服务端把 Accept 循环与信号等待解耦实现优雅停机,客户端用三种消息(短 ASCII、中文、超长)验证长度前缀协议对多字节与粘包的鲁棒性。go run -race main.go 应无输出(无竞态)。扩展:把协议长度从 2 字节升级为 4 字节以支持大消息;为 writeMsg/readMsg 写表驱动单测;尝试用 net.Conn 的 SetKeepAlive 加一层心跳检测。

下一章预告:第9章 并发与异常(重中之重)。网络把 goroutine 推上了舞台,下一章正式登台:CSP 哲学、channel 与 select、Mutex/atomic 与数据竞争检测、没有异常——error 接口 + errors.Is/As/Join + panic/recover,以及 context 的取消与超时。这是 Go 与其他语言最大的分水岭,建议留出整块时间。